Northwestern University

Arithmetic LLM

—— Distributed Version Based on Accelerate

Author: Yizhou Tian

Contents

01 Project Introduction

02 Reconstruction

03 Technical Challenges

04 Summary

01

Project Introduction

- What’s this project?
- Distributed Training
- Accelerate Framework

What is this project?

- A **transformer-based language model** that can evaluate arithmetic expressions with **step-by-step reasoning**

Input: Evaluate: 8 + ((19 + 17) - (15 - 11))

Output:
Step 1: 19 + 17 = 36
Step 2: 15 - 11 = 4
Step 3: 36 - 4 = 32
Step 4: 8 + 32 = 40
Final Result: 40

Distributed Training

Method	Advantages	Disadvantages
Single GPU	Easy code, fast to implement, No communication cost	Slow, memory limit
DP (Data Parallel)	Faster	High communication cost Run on one machine High workload for main GPU
DDP (Distributed Data Parallel)	GPUs are independent Synchronize data	Some communication cost Complicated to apply
Accelerate	Light abstract modularization of DDP	Some communication cost

Accelerator

- accelerate config: Determine machines, devices, precision, etc.
- Accelerator object: an instance that contains all the information needed for DDP training and takes in charge of DDP
- accelerate.prepare(): model, optimizer, train_dataloader = accelerator.prepare(model, optimizer, train_dataloader)
- Launcher: accelerate launch train.py

02

Reconstruction

- Refactoring
- Addition
- Restructuring

Framework-Level Refactoring

- Training loop: reset the optimizer updating steps
- Device management: remove all manual .to(device)
- Loss calculation: aggregate loss globally
- Main process operation: printing and saving

```
# Before: every batch triggers an optimizer step
loss.backward()
optimizer.step()
scheduler.step()

# After: gradient accumulation + distributed sync
with accelerator.accumulate(model):
    accelerator.backward(loss)
    if accelerator.sync_gradients:
        accelerator.clip_grad_norm_(...)
    optimizer.step()
    if accelerator.sync_gradients:
        scheduler.step()
```

Addition

Checkpoint Resuming

- Original code saved optimizer/scheduler states but never restored them -- every restart began from 0 epoch
- Restores: model weights + optimizer momentum + scheduler LR position + epoch/step counters

- Usage:
  - --resume-checkpoint <ckpt_path>

Addition

Early Stopping

- Monitors validation loss improvement ratio (relative, not absolute -- scale-invariant)
- In distributed setting: all processes must agree on when to stop

- Usage:
  - --early-stopping
  - --early-stopping-patience <num>

Restructuring

- Original: flat structure, 40+ files lying in the root
- Present:
  - Modules: training, evaluation, data, model, tools, configs
  - Entry point: full pipeline script, demo
  - Files: README, LISENCE, DEV Log, requirements, etc.

We need to change the path of import and shell script

03

Technical Challenges

- Learning Rate Scheduler
- Early Stopping
- Checkpoints Resumption

Updating Step Calculation

Challenge 1: Updating Step Calculation

- Be careful with the indicators during training even your program doesn’t raise an error
- This bug shows distributed training could increase the stability.
- Fix: changed the way to calculate updating steps and switched from a fixed warmup step `1000` to a ratio-based warmup (5% of total steps)

Early Stopping

Challenge 2: Early Stopping

- Problem: Each GPU get different data, resulting in different parameters and loss, then how should a process stop?

- Deadlock: Process 0 decides to stop (its shard had low loss), but Processes 1-3 continue. At the next synchronizing point or `wait_for_everyone()`, Process 0 hangs because the others never reach that synchronization point.

- Solution: use global loss to determine stop or not

Checkpoint Resumption

Challenge 3: Checkpoint Resumption

- Problem: When interrupted, all progress is lost. Restarting loaded the model weights but reset the optimizer and scheduler, and reset learning rate to initial value

- Solution: let accelerate wrap model, optimizer, and scheduler. Skip the completed epochs when resuming training.

- When train a new model based on existing model, we don’t need to resume the optimizer and scheduler of the existing model.

04

Summary

- Model Result
- Summary

Model Result

Model	Accuracy	Parse Rate
Foundational Model	0.0%	0.0%
Instruction Model	46.4%	88.4%
LoRA Fine-tuned Model	46.6%	87.2%
GRPO Model	68.3%	85.9%

Summary

- Key Technical Contributions
  - Multi-GPU distributed training with HuggingFace Accelerate
  - Distributed-synchronized early stopping and checkpoint resumption
  - Clean, modular project architecture

- How many updating steps you need? When should this updating happens? Where should you place your optimizer and scheduler?
- Global consensus is critical for distributed control flow

- Be careful with the result even though no error is raised
- Use version control (git) wisely and don’t forget to synchronize your code

Q & A

Thank you!

GitHub: github.com/Tian-Yizhou/Arithmetic_LLM